

NAME : K.DAMODHAR

COURSE CODE:

REG.NO:192525266

COURSE NAME:DESIGN AND ANALYSIS

OF ALGORITHMS

Session 3 - Lab Experiments

4. Given an $m \times n$ grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

• Input: $m=2, n=2, N=2, i=0, j=0$ • Output: 6

• Input: $m=1, n=3, N=3, i=0, j=1$ • Output: 12

CODE:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int findPaths(int m, int n, int N, int i, int j) {
```

```
    const int MOD = 1000000007;
```

```
    // dp[r][c] = number of ways to reach cell (r,c)
```

```
    vector<vector<int>> dp(m, vector<int>(n, 0));
```

```
    dp[i][j] = 1;
```

```
    int answer = 0;
```

```
    for (int step = 1; step <= N; step++) {
```

```
        vector<vector<int>> next(m, vector<int>(n, 0));
```

```
for (int r = 0; r < m; r++) {  
    for (int c = 0; c < n; c++) {  
        if (dp[r][c] == 0)  
            continue;  
  
        // Move Up  
        if (r == 0)  
            answer = (answer + dp[r][c]) % MOD;  
        else  
            next[r - 1][c] = (next[r - 1][c] + dp[r][c]) % MOD;  
  
        // Move Down  
        if (r == m - 1)  
            answer = (answer + dp[r][c]) % MOD;  
        else  
            next[r + 1][c] = (next[r + 1][c] + dp[r][c]) % MOD;  
  
        // Move Left  
        if (c == 0)  
            answer = (answer + dp[r][c]) % MOD;  
        else  
            next[r][c - 1] = (next[r][c - 1] + dp[r][c]) % MOD;  
  
        // Move Right  
        if (c == n - 1)  
            answer = (answer + dp[r][c]) % MOD;  
        else
```

```

        next[r][c + 1] = (next[r][c + 1] + dp[r][c]) % MOD;
    }
}

dp = next;
}

return answer;
}

int main() {
    int m, n, N, i, j;

    cout << "Enter m n N i j: ";

    cin >> m >> n >> N >> i >> j;

    cout << "Number of ways = "

        << findPaths(m, n, N, i, j) << endl;

    return 0;
}

```

OUTPUT:

```
Enter m n N i j: Number of ways = 12
```

5. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

CODE:

```
#include <iostream>

#include <vector>

#include <algorithm>

using namespace std;

// Find maximum money from houses in a straight line

int robLinear(vector<int>& nums, int start, int end) {

    int prev2 = 0;

    int prev1 = 0;

    for (int i = start; i <= end; i++) {
```

```

        int current = max(prev1, prev2 + nums[i]);

        prev2 = prev1;

        prev1 = current;
    }

    return prev1;
}

// Find maximum money from circular houses

int rob(vector<int>& nums) {

    int n = nums.size();

    // Only one house
    if (n == 1)

        return nums[0];

    // Case 1: Don't rob the last house
    int case1 = robLinear(nums, 0, n - 2);

    // Case 2: Don't rob the first house
    int case2 = robLinear(nums, 1, n - 1);

    return max(case1, case2);
}

int main() {

    int n;

    cout << "Enter number of houses: ";

    cin >> n;

    vector<int> nums(n);

```

```

    cout << "Enter money in each house: ";

    for (int i = 0; i < n; i++) {

        cin >> nums[i];

    }

    cout << "Maximum money you can rob = " << rob(nums) << endl;

    return 0;

}

```

OUTPUT:

```
Enter number of houses: Enter money in each house: Maximum money you can rob = 11
```

6. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

Input: $n=4$ Output: 5

Input: $n=3$ Output: 3

CODE:

```

#include <iostream>

using namespace std;

int climbStairs(int n) {

    if (n == 1)

        return 1;

```

```
if (n == 2)
```

```
    return 2;
```

```
int a = 1; // ways for 1 step
```

```
int b = 2; // ways for 2 steps
```

```
for (int i = 3; i <= n; i++) {
```

```
    int c = a + b;
```

```
    a = b;
```

```
    b = c;
```

```
}
```

```
return b;
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    cout << "Enter number of steps: ";
```

```
    cin >> n;
```

```
    cout << "Number of distinct ways = " << climbStairs(n) << endl;
```

```
    return 0;
}
```

OUTPUT:

```
Number of distinct ways = 5
```

7. A robot is located at the top-left corner of a $m \times n$ grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

Input: $m=7,n=3$ **Output:** 28

Input: $m=3,n=2$ **Output:** 3

CODE:

```
#include <iostream>

#include <vector>

using namespace std;

int uniquePaths(int m, int n) {
    vector<int> dp(n, 1);

    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
```

```

        dp[j] = dp[j] + dp[j - 1];
    }
}

return dp[n - 1];
}

int main() {
    int m, n;

    cout << "Enter m and n: ";

    cin >> m >> n;

    cout << "Number of unique paths = "

        << uniquePaths(m, n) << endl;

    return 0;
}

```

OUTPUT:

```
Number of unique paths = 28
```